

CYBERSECURITY PORTFOLIO

Cybersecurity Lab Report

Encrypted storage (LUKS + LVM) and preseed automation

Environment: Kali Linux (virtual machine)

Labs included: 2

Date: October 2026

CONTENTS

Lab Overview

This report documents two hands-on labs performed on Kali Linux. Each lab lists its chapter, topic and objective, followed by a step-by-step walkthrough with screenshots.

LAB 1

LUKS-Encrypted LVM Volume

Chapter	Lab 5 (chapter name: add here)
Topic	Encrypted storage with LUKS2, LVM and ext4 on a loop device
Objective	Build an encrypted logical volume on a disk image, mount it, and verify that data can be written and read.

LAB 2

Preseed Configuration and debconf Tools

Chapter	Lab 5 (chapter name: add here)
Topic	Automating Debian/Kali installation answers with a preseed file
Objective	Create a minimal preseed file and install the debconf utilities used to inspect installer settings.

Storage stack built in Lab 1

Image file	Loop device	LUKS container	LVM (PV, VG, LV)	ext4	Mount point
luks-lab-f.img	/dev/loop0	luks_lab	lab_vg / lab_lv	filesystem	/mnt/luks-lab

LAB 1

LUKS-Encrypted LVM Volume on a Loop Device

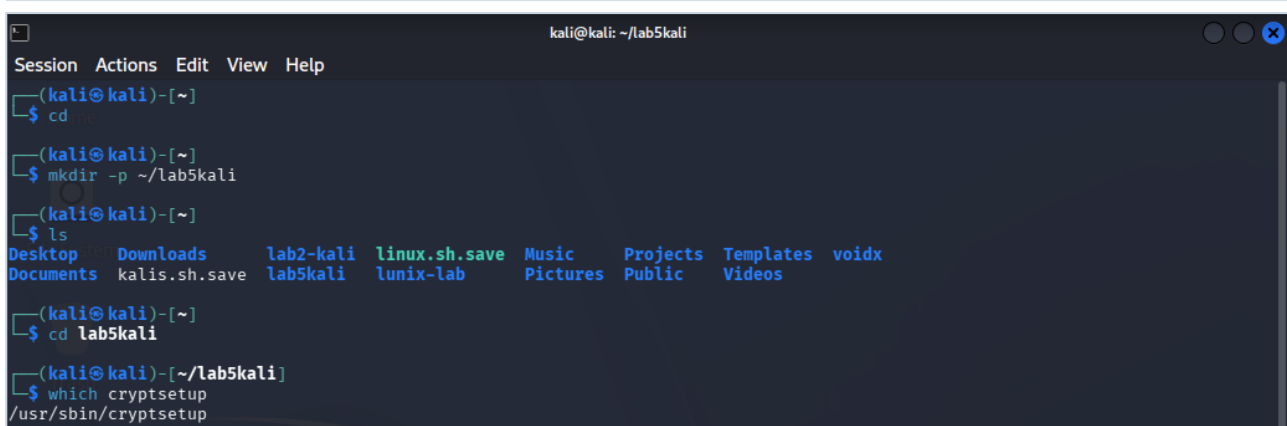
Chapter: Lab 5 | Topic: Full-volume encryption, LVM, ext4

Objective

Build an encrypted logical volume on a 500 MB disk image (image file, loop device, LUKS, LVM, ext4), mount it and verify it works. Environment: Kali Linux VM.

Procedure

1 Prepare the workspace



```
kali@kali: ~/lab5kali
Session Actions Edit View Help
(kali@kali)-[~]
$ cd
(kali@kali)-[~]
$ mkdir -p ~/lab5kali
(kali@kali)-[~]
$ ls
Desktop  Downloads  lab2-kali  linux.sh.save  Music  Projects  Templates  voidx
Documents  kalis.sh.save  lab5kali  linux-lab  Pictures  Public  Videos
(kali@kali)-[~]
$ cd lab5kali
(kali@kali)-[~/lab5kali]
$ which cryptsetup
/usr/sbin/cryptsetup
```

Lab directory created and cryptsetup located.

Created ~/lab5kali and confirmed cryptsetup is installed.

2 Attach the image to a loop device

```
(kali@kali)~/lab5kali
$ sudo losetup -d /dev/loop0
(kali@kali)~/lab5kali
$ sudo losetup -d /dev/loop1
(kali@kali)~/lab5kali
$ sudo losetup -d /dev/loop2
(kali@kali)~/lab5kali
$ sudo losetup -d /dev/loop3
(kali@kali)~/lab5kali
$ sudo losetup -a
(kali@kali)~/lab5kali
$ sudo losetup --find --show luks-lab-f.img
dev/loop0
(kali@kali)~/lab5kali
$ sudo losetup -a
```

Old loop devices detached; image attached as /dev/loop0.

```
(kali@kali)~/lab5kali
$ sudo losetup -a
dev/loop0: [2049]:467950 (/home/kali/lab5kali/luks-lab-f.img)
(kali@kali)~/lab5kali
$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
loop0 7:0 0 500M 0 loop
sda 8:0 0 20G 0 disk
sda1 8:1 0 18.9G 0 part /
sda2 8:2 0 1K 0 part
sda5 8:5 0 1.1G 0 part [SWAP]
r0 11:0 1 1024M 0 rom
```

Only loop0 (500 MB) remains.

Losetup turns the image file into a block device, /dev/loop0, so it can be encrypted like a real disk.

3 Format the device with LUKS

```
(kali@kali)~/lab5kali
$ sudo cryptsetup luksFormat /dev/loop0
WARNING!
This will overwrite data on /dev/loop0 irrevocably.
Are you sure? (Type 'yes' in capital letters): YES
Enter passphrase for /home/kali/lab5kali/luks-lab-f.img:
Verify passphrase:
```

Confirmation prompt, then passphrase and verification.

luksFormat writes the LUKS header and sets the passphrase. It erases existing data, so it asks for confirmation.

4 Verify the LUKS container

```
(kali@kali)~/lab5kali
$ sudo cryptsetup isLuks /dev/loop0
(kali@kali)~/lab5kali
$ echo $?
0
```

Exit status 0 = valid LUKS device.

isLuks answers through its exit code; 0 means success.

5

Inspect the LUKS header

```
(kali@kali)-[~/lab5kali]
└─$ sudo cryptsetup luksDump /dev/loop0
LUKS header information
Version:          2
Epoch:           3
Metadata area:    16384 [bytes]
Keyslots area:    16744448 [bytes]
UUID:             87a797fd-577a-4bff-86ef-8d27946fac83
Label:            (no label)
Subsystem:        (no subsystem)
Flags:            (no flags)

Data segments:
 0: crypt
   offset: 16777216 [bytes]
   length: (whole device)
   cipher: aes-xts-plain64
   sector: 512 [bytes]

Keyslots:
 0: luks2
   Key:        512 bits
   Priority:    normal
   Cipher:     aes-xts-plain64
   Cipher key: 512 bits
   PBKDF:      argon2id
   Time cost:  4
   Memory:     507341
   Threads:    1
   Salt:       70 c4 b5 d0 91 88 ed ae 38 93 07 79 14 67 e6 1d
              1a 07 57 98 43 c2 59 5d b6 3f 44 0e 6d df 8d 32
   AF stripes: 4000
   AF hash:    sha256
   Area offset:32768 [bytes]
   Area length:258048 [bytes]
   Digest ID:  0

Tokens:
Digests:
 0: pbkdf2
   Hash:       sha256
   Iterations: 567411
   Salt:       e6 94 27 67 8a 49 1b da 33 d9 ba 32 54 62 4c 6b
              f4 ba 12 03 80 d1 13 79 cf 9a 2f 26 34 1c 06 a8
   Digest:     0e c5 a5 34 ba 45 74 01 94 fb c8 ce 3a fb 6b dc
              e9 e1 f0 ad 7b c6 8e 17 2b 90 78 72 ce b8 99 a1
```

LUKS2 header details.

The header confirms LUKS2 with `aes-xts-plain64`, a 512-bit key and `argon2id` for the passphrase key derivation.

6 Unlock and create the LVM layers

```
(kali@kali)~/lab5kali
$ sudo cryptsetup open /dev/loop0 luks_lab
Enter passphrase for /home/kali/lab5kali/luks-lab-f.img:

(kali@kali)~/lab5kali
$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE  MOUNTPOINTS
loop0        7:0      0   500M 0 loop
luks_lab    254:0     0    484M 0 crypt
sda          8:0      0    20G 0 disk
sda1         8:1      0   18.9G 0 part /
sda2         8:2      0     1K 0 part
sda5         8:5      0    1.1G 0 part [SWAP]
sr0         11:0     1   1024M 0 rom

(kali@kali)~/lab5kali
$ sudo pvcreate /dev/mapper/luks_lab
Physical volume "/dev/mapper/luks_lab" successfully created.

(kali@kali)~/lab5kali
$ sudo pvs
PV                VG Fmt Attr PSize  PFree
/dev/mapper/luks_lab  lvm2 --  484.00m 484.00m

(kali@kali)~/lab5kali
$ sudo vgcreate lab_vg /dev/mapper/luks_lab
Volume group "lab_vg" successfully created

(kali@kali)~/lab5kali
$ sudo lvcreate -l 100%FREE -n lab_lv lab_vg
Logical volume "lab_lv" created.

(kali@kali)~/lab5kali
$ sudo lvs
LV      VG      Attr      LSize   Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
lab_lv  lab_vg  -wi-a----- 480.00m
```

Unlocked as luks_lab; PV, VG and LV created.

The container is unlocked as /dev/mapper/luks_lab, then turned into a physical volume, volume group lab_vg and logical volume lab_lv (about 480 MB).

7

Create the filesystem and mount

```
(kali@kali)~[/lab5kali]
$ sudo pvs
PV                VG      Fmt  Attr  PSize  PFree
/dev/mapper/luks_lab lab_vg  lvm2 a--   480.00m  0

(kali@kali)~[/lab5kali]
$ sudo vgs
VG      #PV #LV #SN Attr   VSize  VFree
lab_vg   1   1   0 wz--n- 480.00m  0

(kali@kali)~[/lab5kali]
$ sudo mkfs.ext4 /dev/lab_vg/lab_lv
mke2fs 1.47.4 (6-Mar-2025)
Creating filesystem with 491520 1k blocks and 122880 inodes
Filesystem UUID: a83c1b0a-29dd-40f6-bfb5-ea1e012535af
Superblock backups stored on blocks:
    8193, 24577, 40961, 57345, 73729, 204801, 221185, 401409

Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done

(kali@kali)~[/lab5kali]
$ sudo blkid /dev/lab_vg/lab_lv
/dev/lab_vg/lab_lv: UUID="a83c1b0a-29dd-40f6-bfb5-ea1e012535af" BLOCK_SIZE="1024" TYPE="ext4"

(kali@kali)~[/lab5kali]
$ sudo mkdir -p /mnt/luks-lab

(kali@kali)~[/lab5kali]
$ sudo mount /dev/lab_vg/lab_lv /mnt/luks-lab

(kali@kali)~[/lab5kali]
$ df -h /mnt/luks-lab
Filesystem              Size  Used Avail Use% Mounted on
/dev/mapper/lab_vg-lab_lv 440M  494K  412M   1% /mnt/luks-lab

(kali@kali)~[/lab5kali]
$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE  MOUNTPOINTS
loop0         7:0    0   500M  0 loop
luks_lab     254:0    0   484M  0 crypt
lab_vg-lab_lv 254:1    0   480M  0 lvm    /mnt/luks-lab
sda           8:0    0    20G  0 disk
├─sda1         8:1    0   18.9G  0 part  /
├─sda2         8:2    0    1K  0 part
├─sda5         8:5    0    1.1G  0 part  [SWAP]
sr0          11:0    1   1024M  0 rom
```

ext4 created and mounted at /mnt/luks-lab.

The logical volume is formatted as ext4 and mounted. `lsblk` shows the full stack: `loop0`, `luks_lab` (crypt), `lab_vg-lab_lv` (lvm), mount point.

8

Test the volume

```
(kali@kali)~[/lab5kali]
$ sudo touch /mnt/luks-lab/test.txt

(kali@kali)~[/lab5kali]
$ ls -l /mnt/luks-lab
total 12
drwx----- 2 root root 12288 Oct  8 20:18 lost+found
-rw-r--r-- 1 root root    0 Oct  8 20:21 test.txt

(kali@kali)~[/lab5kali]
$ echo "luks +vmlab" | sudo tee /mnt/luks-lab/test.txt
luks +vmlab

(kali@kali)~[/lab5kali]
$ cat /mnt/luks-lab/test.txt
luks +vmlab
```

test.txt written and read back.

Root owns the mount point, so `sudo` is used; `| sudo tee` writes the file as root. The file reads back correctly.

Key Concepts

Term	Meaning
Loop device	A file used as a block device.
LUKS	Linux disk-encryption standard.

Term	Meaning
LVM	Pools storage (PV, VG) and carves flexible logical volumes (LV).
ext4	Journaling filesystem on the logical volume.

Results and Takeaways

- The image was encrypted with LUKS2, layered with LVM, formatted as ext4 and mounted.
- A test file was written and read from the encrypted volume.
- A device needs a LUKS header before it can be opened; encryption sits below LVM and the filesystem.

LAB 2

Preseed Configuration and debconf Tools

Chapter: Lab 5 (preslab) | Topic: Unattended installation answers

Objective

Create a basic preseed file and use the debconf tools to look at installer settings. Environment: Kali Linux VM, nano.

Procedure

1 Create the preseed file

```
d-i mirror/suite string kali-rolling
d-i passwd/root-login boolean false
d-i passwd/user-fullname string Lab User
d-i passwd/username string labuser
```

Final preseed.cfg content.

A preseed file answers installer questions in advance. Here it uses kali-rolling, disables root login and creates the user labuser.

2 Install the debconf utilities

```
(kali@kali)~[/lab5kali/preslab]
$ sudo apt install debconf-utils
The following packages were automatically installed and are no longer required:
  iucode-tool      python3.13      python3.13-minimal  python3.13-venv
  libpython3.13-dev python3.13-dev  python3.13-tk      rpcsvc-proto
Use 'sudo apt autoremove' to remove them.

Installing:
  debconf-utils

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 1438
  Download size: 57.0 kB
  Space needed: 111 kB / 2,704 MB available

Get:1 http://kali.download/kali kali-rolling/main amd64 debconf-utils all 1.5.92 [57.0 kB]
Fetched 57.0 kB in 6s (9,489 B/s)
Selecting previously unselected package debconf-utils.
(Reading database... 426222 files and directories currently installed.)
Preparing to unpack .../debconf-utils_1.5.92_all.deb...
Unpacking debconf-utils (1.5.92)...
Setting up debconf-utils (1.5.92)...
Processing triggers for man-db (2.13.1-1)...
Processing triggers for kali-menu (2026.2.6)...

(kali@kali)~[/lab5kali/preslab]
$ which debconf-get-selections
/usr/bin/debconf-get-selections
```

debconf-utils installed.

debconf-utils provides debconf-get-selections, which exports debconf answers in preseed format.

3

Query the installer selections

```
(kali㉿kali)-[~/lab5kali/preslab]
└─$ debconf-get-selections --installer
debconf: DbDriver "passwords" warning: could not open /var/cache/debconf/passwords.dat: Permission denied
debconf: DbDriver "di_questions": could not open /var/log/installer/cdebconf/questions.dat: Permission denied
```

Permission denied on the debconf data files.

The debconf databases are root-protected, so a normal user cannot read them. Reading this data requires elevated privileges.

Results and Takeaways

- A four-line preseed file was created.
- debconf-utils was installed.
- Preseed lines follow the form `d-i question type value`.
- debconf data is protected by file permissions.